

Evaluating LLM Agents on Automated Software Analysis Tasks

Islem Bouzenia
bouzenia.islem@pm.me
CISPA Helmholtz Center for
Information Security
Germany

Cristian Cadar
c.cadar@imperial.ac.uk
Imperial College London
United Kingdom

Michael Pradel
michael@binaervarianz.de
CISPA Helmholtz Center for
Information Security
Germany

Abstract

Numerous software analysis tools exist today, yet applying them to diverse open-source projects remains challenging due to environment setup, dependency resolution, and tool configuration. LLM-based agents offer a potential solution, yet no prior work has systematically studied their effectiveness on the specific task of *automated software analysis*, which, unlike issue solving or general environment setup, requires installing and configuring a separate analysis tool alongside the target project, generating tool-specific prerequisites, and validating that the tool produces meaningful analysis outputs. We introduce AnalysisBench, a benchmark of 35 tool-project pairs spanning seven analysis tools and ten diverse C/C++ and Java projects, each with a manually constructed reference setup. Using AnalysisBench, we evaluate four agent architectures across four LLM backends. Our custom agent, AnalysisAgent, achieves *manually verified* success rates of 94% (Gemini-3-Flash, 33/35 tasks), compared to 77% for the best baseline (ExecutionAgent). Beyond quantitative results, we identify key limitations in existing agents, including stage mixing, poor error localization, and premature termination, and show that agentic architecture matters more than LLM capability alone. We further find that whole-program analyses and symbolic execution are the most difficult tasks, that Java toolchains pose greater challenges than C/C++, and that LLM-self-validated success consistently overstates manually verified success.

ACM Reference Format:

Islem Bouzenia, Cristian Cadar, and Michael Pradel. 2026. Evaluating LLM Agents on Automated Software Analysis Tasks. In . ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

To support developers in ensuring software quality, security, and performance, decades of research and engineering have produced a wide variety of *software analysis tools*. For example, developers can choose from a diverse range of static analyzers [2, 20, 57], symbolic execution engines [7, 48, 52], fuzzers [17, 66], and profilers [12, 21, 23, 61]. When adopted widely, these tools have a proven impact: For instance, Google’s static analysis platform Tricorder surfaces hundreds of thousands of warnings per day across the company’s

codebase [47], and OSS-Fuzz has found thousands of vulnerabilities in open-source projects through continuous fuzzing [9].

Unfortunately, applying a given analysis tool to a new project is notoriously difficult [3]. One part of this challenge is creating a compatible execution environment for the analysis tool itself, which may require a specific compiler version, complex dependencies, and non-trivial configuration. Another part of the challenge is preparing the target project for analysis, which may involve building the project with special flags, generating tool-specific prerequisites (e.g., LLVM bitcode), or launching a runtime environment (e.g., JVM) with specific parameters. Typically, developers want to perform these tasks in an isolated, containerized environment to avoid polluting their local setup and ensure reproducibility. This environment needs to be compatible with both the analysis tool and the target project, which adds further complexity. Finally, developers must validate that the analysis actually ran and produced meaningful outputs (e.g., warnings, test cases, call graphs), as a mis-configured setup may skip most or all of the project’s code and give a false sense of success. Due to these challenges, adoption of analysis tools in practice remains limited. Prior work has documented that the usability and setup burden of static analyzers is a major barrier to their adoption [3, 32], and similar observations have been made for fuzzers [17] and symbolic execution engines [7].

Given the multi-step, environment-intensive nature of automated software analysis, LLM-based agents are a natural fit: they can plan and execute shell commands, interpret error messages, and iteratively adapt to unforeseen failures. Such agents have already demonstrated success in related tasks, including issue solving [28, 60, 62, 67], automated program repair [5, 10, 45], web automation [60], and environment construction [4, 6, 16, 41]. However, end-to-end execution of software analysis tools remains underexplored. Unlike project-only environment setup [6], automated software analysis requires the agent to install and configure a separate analysis tool alongside the target project, produce tool-specific prerequisites (e.g., LLVM bitcode, Java bytecode), and validate that the tool produces meaningful analysis outputs rather than merely running without errors. As we show in our evaluation, simply changing the task description in an existing agent yields substantially lower success rates than a purpose-built agentic architecture (e.g., the best baseline averages 57% verified success across LLM backends vs. 79% for AnalysisAgent, a gap of 20 percentage points).

To systematically study the capabilities and limitations of LLM-based agents on this task, we construct *AnalysisBench*, the first benchmark for evaluating LLM-based agents on end-to-end *automated software analysis*. AnalysisBench evaluates an agent’s ability to set up and execute diverse analysis tools on open-source projects from scratch, spanning the full pipeline from environment construction to evidence validation. The benchmark consists of 35

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference’17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

tool-project pairs spanning seven analysis tools and ten diverse projects in C/C++ and Java (Tables 1 and 2). Each task includes a manually verified reference setup and success criteria, enabling rigorous evaluation.

We compare three baselines, RAG-Agent [34], Mini-SWE-Agent [62], and ExecutionAgent [6], across four LLMs (GPT-5-nano, GPT-5-mini, DeepSeek-V3.2, Gemini-3-Flash). This comparison reveals both promise and systematic limitations in existing agents: *stage mixing* (interleaving unrelated workflow steps, e.g., attempting analysis before the tool is installed), *poor error localization* (verbose logs obscuring root causes), and *premature termination* (§3.2), with the best baseline (ExecutionAgent + Gemini-3-Flash) reaching 77% verified success.

To address these limitations, we present *AnalysisAgent*, a novel agent designed specifically for the automated software analysis task. Our approach incorporates three design principles that each address one of the above limitations: explicitly staged execution, single-action cycles with log condensation, and evidence-based validation. Empirical evaluation across four LLM backends shows that AnalysisAgent with Gemini-3-Flash achieves the highest verified success rate across all configurations (94%, 33/35 tasks). Because failed runs consume $2.77\times$ more cycles and cost $1.27\times$ more than successful runs, the higher success rate of AnalysisAgent also lowers the overall cost profile by reducing failures.

This paper makes the following contributions:

- **Task and benchmark.** We formalize automated software analysis and present AnalysisBench, a benchmark of 35 tool-project pairs (seven tools, ten projects, C/C++ and Java) with manually constructed reference setups.
- **Empirical study.** We evaluate four agentic architectures across four LLMs, analyzing success rates, efficiency, and failure modes.
- **Agent design.** We present AnalysisAgent, combining staged execution, single-action cycles with log condensation, and evidence-based validation, and show empirically that this task-specific design matters beyond LLM capability alone.

We release the benchmark, agents, and data: <https://anonymous.4open.science/r/agents-for-analysis-8AE4>.

2 AnalysisBench

2.1 Task Definition

The *automated software analysis* task is about automatically applying a software analysis tool to a real-world software project. The task is to be performed end-to-end, i.e., starting from a fresh environment and finishing with verifiable evidence that a meaningful analysis has been successfully completed.

Inputs. Each task is defined by a tool-project pair (T, P) and an execution interface.

- **Tool specification T .** Tool name and acquisition method (e.g., upstream repository URL or a released package).
- **Target project P .** Repository URL and pinned revision (commit hash or tag) of the target project to be analyzed.
- **Execution interface.** A writable workspace, environment interaction tools (e.g., terminal), and a budget (5 h, \$2 per task) ensuring comparable evaluation.

Table 1: Analysis tools in AnalysisBench.

Tool	Lang.	Technique	Output
AFL++ [17]	C/C++	Fuzzing	Coverage, queued test cases, exercised inputs, hangs and crashes.
KLEE [7]	C/C++	Symbolic execution	Generated tests (.ktest), path exploration statistics (executed instructions, coverage...)
CSA [11]	C/C++	Static analysis	Structured reports with source-level warnings and bugs found.
cflow [19]	C	Structural analysis	Text-based call graphs from C sources.
Infer [8]	Java	Static analysis	Issue reports (infer-out/) from whole-program analysis.
WALA [54]	Java	Static analysis	Call graphs and pointer-analysis results from bytecode.
SJK [53]	Java	Profiling	Thread dumps, heap histograms, sampling profiles via JVM attach.

Terminology. Throughout this paper, we use *artifacts* to refer to concrete build and analysis outputs (e.g., bitcode files, analysis reports), and *evidence* to refer to artifacts that demonstrate meaningful analysis completion on the target project.

Task objective. Given (T, P) , the agent must (i) provision an isolated Docker container, (ii) install the tool and its dependencies, (iii) fetch and build (when required) the target project, (iv) execute the analysis tool on the target project, and (v) emit evidence that the analysis tool processed project-specific inputs (e.g., source files) and produced tool-specific output artifacts (e.g., warnings) beyond trivial outputs such as tool version strings or help messages.

Outputs. A successful run produces an evidence package:

- **Reproducible environment.** E.g., a Dockerfile
- **Complete execution trace.** The full transcript of executed commands, including stdout/stderr, return codes, and key environment metadata (tool version, OS image, commit hashes).
- **Tool-specific analysis artifacts.** Logs and output files from executing T on P (e.g., warnings, test cases, call graphs).

Evidence-based success criteria. We consider a task solved successfully if the environment is reproducible, the tool is installed and runnable, the target project is built and prepared, the tool is invoked on project-relevant inputs, and the outputs contain verifiable, project-specific analysis evidence (e.g., warnings referencing project paths, generated test cases, or call graphs). We operationalize these criteria through tool-specific validation checks detailed in §2.3 and discuss limitations in §5.

2.2 Benchmark Construction

To evaluate agents on the automated software analysis task, we construct AnalysisBench, a new benchmark of 35 tool-project pairs.

2.2.1 Analysis Tools. Table 1 summarizes the seven analysis tools included in AnalysisBench and the types of analyses enabled through these tools. We select tools based on three criteria: (i) they are open-source and freely available, (ii) they represent distinct analysis techniques (fuzzing, symbolic execution, static analysis,

Table 2: Build systems and dependencies in AnalysisBench.

Repository	Build toolchain	Key dependencies
<i>Analysis tools:</i>		
AFL++	Make; gcc/clang	LLVM/clang (opt.)
KLEE	CMake	LLVM; STP/Z3; klee-uclibc
CSA	CMake + Ninja/Make	LLVM; zlib (opt.)
cflow	Autotools + Make	Standard C library
Infer	Buck + opam	OCaml; JDK; Clang/LLVM
WALA	Gradle/Maven	JDK
SJK	Maven	JDK
<i>C/C++ projects:</i>		
curl	Autotools; CMake	OpenSSL, zlib, nghttp2
ImageMagick	Autotools; CMake	libpng, libjpeg, zlib
fastfetch	CMake	System libraries
masscan	Make	libpcap
radare2	Meson	capstone, xxhash
<i>Java projects:</i>		
Tika	Maven	JDK; parser libraries
Closure Compiler	Bazel/Maven	JDK
Saxon-HE	Maven	JDK
JMH	Maven	JDK
Checkstyle	Maven	JDK

structural analysis, and profiling), and (iii) they require non-trivial setup beyond a simple package install, which is the core challenge our benchmark targets. However, we acknowledge that we do not cover all important tools (e.g., Valgrind, AddressSanitizer, SpotBugs).

2.2.2 Target projects. The project sections of Table 2 show the ten selected projects, split between five C/C++ and five Java codebases. We select target projects that are actively maintained, and cover diverse build systems and dependencies. To find candidates, we search GitHub for popular repositories using keywords such as “command-line”, “program”, and “cmd”, sorting results by star count. We specifically target *command-line programs* that can be invoked directly after installation, because they provide well-defined entry points for analysis tool invocation and unambiguous validation criteria. We exclude libraries or frameworks typically used only as dependencies or submodules in other projects. We acknowledge that this restriction may favor projects with well-defined entry points, and that analyzing libraries (which often lack standalone executables) presents additional challenges, such as harness construction. We discuss this threat to generalizability in §5. For each candidate, we manually inspect the README and documentation to verify the presence of clear command-line invocation examples.

2.2.3 Tool-Project Pairs. To construct the tool-project tasks in AnalysisBench, we pair each tool with the projects to which it is applicable: four C/C++ tools paired with five C/C++ projects ($four \times five = 20$ tasks), and three Java tools paired with five Java projects ($three \times five = 15$ tasks), for a total of 35 tasks. Each task is executed independently in a fresh container; for the same project, the tool-specific stages can differ substantially across tools (§4.2).

2.3 Validation of Success

A central challenge in evaluating agentic automation of software analysis is distinguishing genuine end-to-end analysis from superficial “success” signals. For example, an agent might install an analysis tool, run its `-help` command, and then declare success without actually analyzing the target project. To thoroughly check for meaningful analysis, we manually construct reference artifacts and validation mechanisms for each tool-project pair.

Reference environment construction. For each (T, P) pair, we manually construct a containerized reference setup that installs T , clones P , performs the required build steps, and runs the analysis under the benchmark budget.

Reference artifacts. We execute each reference setup to produce a reference evidence package: full logs, key intermediate build outputs, and tool-specific analysis artifacts (e.g., static analysis reports, call graphs, generated tests, fuzzing queues/crashes, or JVM diagnostic snapshots). These artifacts define what a meaningful execution looks like for the (T, P) pair.

Validation process. When an agent completes a task, we re-execute the agent’s produced setup (e.g., Dockerfile and scripts) to confirm reproducibility, and compare the agent’s artifacts against the reference evidence package. To further validate the analysis run, we may extend the analysis beyond the limit the agent has chosen. Outlier or suspicious values (e.g., unusually low coverage, empty reports, or results inconsistent with the reference) are flagged and discussed among authors. To support reproducibility, our replication package includes the detailed validation protocol.¹ Specifically, we check:

- *Structural evidence:* Expected files and directories exist (KLEE’s `klee-out` directory, AFL++’s `queue` directory, etc.).
- *Project references:* Logs or reports reference project-specific paths, symbols, or build artifacts.
- *Semantic evidence:* Tool-specific indicators of analysis progress, such as partial or completed path exploration, AFL coverage growth, generated call graphs, or static analysis warnings.

This reference-backed validation supports human auditing and helps spot premature termination by requiring artifacts that are difficult to produce without running analysis on the target repository.

3 Agents for Automated Analysis

To study the extent to which LLM-based agents can perform the automated software analysis task defined in §2, we evaluate four agent architectures on AnalysisBench. We first describe three baseline agents adapted from prior work (§3.1), then analyze their common failure modes (§3.2), and finally introduce AnalysisAgent, a custom agent designed to address these limitations (§3.3). Across all agents, we provide the same task specification (§2) and the same budget (5 h and \$2 per task). The key difference is how each agent plans and executes its actions and when it decides to stop. All baseline agents rely on the LLM’s own judgment to decide when to stop, without explicit output validation checks.

¹https://anonymous.4open.science/r/agents-for-analysis-8AE4/MANUAL_VERIFICATION_PROTOCOL.md

3.1 Baseline Agents

RAG-Agent. Retrieval-augmented generation (RAG) is a common paradigm to enhance LLM capabilities by retrieving external knowledge [34]. We re-implement this paradigm for the automated software analysis task in *RAG-Agent*, following the design of a RAG agent described by LangChain [15] but using LiteLLM to support multiple LLM backends. We include *RAG-Agent* to represent the “plan-then-execute” paradigm that is common in LLM-based automation pipelines. Unlike the other agents, *RAG-Agent* does not interactively debug inside the container; instead, it synthesizes complete scripts upfront and can only revise them based on execution feedback, without fine-grained, command-level interaction. The agent first issues web queries to collect installation and invocation guidance, e.g., in the form of tool READMEs, GitHub issues, and tutorials. Then, the agent synthesizes executable artifacts: a Dockerfile, a setup script that installs dependencies and builds the project, and a launch script that runs the analysis.

Adaptation. We seed retrieval with the tool name, its upstream repository, the project name, and its repository URL.

Mini-SWE-Agent. An agent architecture designed for software engineering tasks [38, 62]. Instead of producing a single large script, it alternates between reasoning and tool actions (shell commands, file edits), updating its plan based on observed errors. This style is well suited to incremental debugging and refinement. While conceptually simple, Mini-SWE-Agent has demonstrated strong performance on software engineering tasks and is one of the leading openly available agents on SWE-bench Verified [31].

Adaptation. We modify the prompt to specify the automated software analysis task.

ExecutionAgent. ExecutionAgent [6] is an environment-setup agent designed to build a given project and execute its test suite in a container environment. Compared to Mini-SWE-Agent, it prioritizes creating a runnable container specification and then executing the workload within that environment.

Adaptation. We disable ExecutionAgent’s test-specific retrieval mechanisms (e.g., test-suite discovery and CI workflow parsing), which are not applicable to the analysis task.

3.2 Limitations and Failure Trends

While the baselines differ in planning and execution style, early experiments reveal three common limitations when applied to automated software analysis.

1) Missing procedural structure leads to stage mixing. All baselines occasionally attempt operations in the wrong order, such as cloning a repository into a non-existent container context, invoking analyzers before the tool’s dependencies are installed or before a build artifact of the project exists. Because error messages are often locally scoped (e.g., missing binary, missing path, missing library), agents frequently respond with additional installation attempts instead of correcting the underlying ordering mistake.

2) Verbose logs obscure root causes, impeding recovery. Long logs produced when executing commands may obscure the root cause of a failure. *RAG-Agent* in particular suffers from this problem due to its monolithic scripts, but Mini-SWE-Agent and ExecutionAgent also occasionally face this challenge. In all cases, poor error localization reduces the agent’s ability to form targeted hypotheses (e.g., “the project uses CMake but container is missing ninja-build”), wasting cycles on uninformed trial and error.

3) Weak success validation causes premature termination. All baselines sometimes stop after observing superficial signals of success. Common examples include:

- *Tool presence only:* Commands like `klee --version` or `infer --help` succeed, and hence, the agent declares success.
- *Toy execution:* The tool runs on a trivial input (e.g., a small file or example program), but not on the target project.
- *Partial pipeline:* The project builds successfully, but the analysis is skipped or misconfigured, producing no tool-specific results.

Example 1: Stage mixing and environment thrashing (Mini-SWE-Agent, CSA + curl, Gemini-3-Flash). Mini-SWE-Agent correctly reports 3 warnings, yet the run is recorded as a failure. Stage mixing causes the agent to interleave package installation with analysis attempts, wasting 17 of 25 iterations on `dpkg` lock contention from concurrent package-manager invocations. Although the analysis does complete, a stale Docker-cleanup status message misleads the agent into believing the environment has failed, so it never recognizes its own success. This example illustrates how *stage mixing* (limitation 1) wastes cycles and how *weak success validation* (limitation 3) causes the agent to discard valid results.

Example 2: Repetitive cycling without progress (RAG-Agent, WALA + Checkstyle, DeepSeek-V3.2). RAG-Agent uses all 120 iterations trying to build WALA from source (Maven, Gradle, different JDKs, hallucinated version tags) without ever diagnosing the root cause (missing C++ toolchain for WALA’s cast module). This example illustrates *poor error localization* (limitation 2).

3.3 AnalysisAgent

Motivated by these limitations, we introduce AnalysisAgent, a custom agent built on three principles: staged execution, single-action cycles with log condensation, and evidence-based validation.

3.3.1 Principle 1: Explicitly Staged Execution. The agent follows a four-stage linear progression with per-stage instructions:

- (1) *Docker setup.* Writes a Dockerfile and auxiliary files. No container exists yet, so terminal commands are disabled. The framework enforces a tool whitelist, allowing only `write_to_file` and `read_file`; any other action is rejected. The agent designs a compatible base image and installs build dependencies.
- (2) *Tool setup.* The framework automatically advances to this stage once the container is running and responds to a shell command. In this stage, we instruct the agent to install the analysis tool and conduct a successful smoke test of the analysis tool, e.g., by invoking it on a small toy example or showing `--help`.
- (3) *Project setup.* We give the agent instructions to build the project and generate tool-specific artifacts (e.g., compilation database

for static analyzers or bitcode for KLEE). We instruct the agent to check the availability of such artifacts before switching stage.

- (4) *Analysis run*. In this stage, the agent is instructed to apply the installed tool to the project (i.e., not only toy examples) and provide the outputs in a results directory. When the agent declares the task complete, an LLM-as-judge validates the claim; if the judge rejects, execution continues if cycles remain (Principle 3).

The transition from Stage 1 to Stage 2 is automatic: the framework advances once the Docker container is running and responsive. Between later stages (2→3, 3→4), the agent determines when stage goals are met based on its assessment of whether progress has been made as specified in the prompt.

In each stage, the prompt carries over a summary of what happened in previous stages (e.g., which versions of the tool and its dependencies were installed, compilation results, preview of latest actions). The carry-over information contains recent commands from different stages and a synthesized summary obtained by a separate LLM query. While staged pipelines are common in CI/CD, our decomposition is tailored to automated software analysis.

3.3.2 Principle 2: Single-Action Cycles with Deterministic Log Condensation. Weaker LLMs struggle to express correctly formatted commands. AnalysisAgent therefore triggers actions in two steps. First, the agent receives stage context, recent observations, and structured feedback from prior failures, and outputs free-form reasoning with proposed next steps. Second, a separate LLM call extracts the first concrete action from this response, formats it as an executable command (following a per-tool JSON schema that we define for each agentic tool), and the system executes it, returning the observation (stdout/stderr, exit code, environment metadata) for the next cycle. Invalid actions (e.g., stopping the container, host-to-container copies) are rejected by the framework with an error message returned to the agent, and the cycle is consumed without effect. As build systems and analysis tools may produce thousands of lines of output, we need to condense outputs. Unlike prior work that relies on LLM-based summarization [6], AnalysisAgent uses deterministic pattern matching to extract diagnostic signals: the first 1,000 characters (command + initial progress), lines matching any of 70 manually curated failure patterns (e.g., compiler/linker errors, missing files, package errors), and the last 1,000 characters (exit status + summary). The action-observation loop follows the ReAct paradigm [64]; the key differences are the separation of planning from extraction and the deterministic log condensation, both tailored to the long, noisy outputs of build systems and tools.

3.3.3 Principle 3: Evidence-Based Validation. When the agent declares the task complete, the system constructs an *evidence package* comprising three components: (1) stage summaries (successful/failed commands, environment choices), (2) recent analysis-stage observations, and (3) output file locations. This package is then submitted to an independent *LLM-as-judge*. The judge uses the same LLM backend as the agent. To mitigate circular bias, the validator operates on the structured package (not raw conversation history), uses a separate prompt focused solely on output validation, and has no access to the agent’s reasoning or conversation context. The judge verifies that (i) the invocation targets the specified project and (ii) output artifacts consistent with tool completion exist (e.g., KLEE

```
$ klee --posix-runtime --libc=uclibc \
--max-time=300 --max-memory=1024 \
--only-output-states-covering-new \
--emit-all-errors --output-dir=klee-out \
./fastfetch.bc --sym-args 0 2 8 --sym-stdin 32
```

Figure 1: Stage 4 (analysis run) for KLEE on fastfetch.

test cases, AFL++ crash queues). It then compares the actual output against a reference example synthesized by an LLM from the tool’s official documentation, generated once per tool and reused across all tasks for that tool, helping exclude trivial or error-produced output. If validation fails, the system notifies the agent of the rejection reason and triggers continued execution (if cycles remain). This LLM-as-judge serves as an internal heuristic to reduce premature termination during agent execution; it is not the final evaluation metric. Across all AnalysisAgent runs, the judge accepted 131 of 140 submissions as successful, of which 111 were confirmed by manual verification, yielding a false-positive rate of 15%. While LLM-as-judge approaches have been used for general-purpose evaluation [68], the structured evidence package and tool-specific reference comparison specifically target the challenge of distinguishing genuine analysis completion from superficial success signals.

3.3.4 Example Run: KLEE on fastfetch. We illustrate the workflow with a concrete task: running the KLEE symbolic execution engine on fastfetch, a C-based system information tool.

Stage 1: Docker setup. The agent generates a Dockerfile based on ubuntu:22.04 with a minimal build toolchain (Git, CMake, Clang, and related utilities). The framework builds the image and starts a container; once the shell responds, the agent transitions to Stage 2.

Stage 2: Tool setup. Inside the container, the agent detects that KLEE is absent, installs LLVM-14 and Z3, builds klee-uclibc, then builds and installs KLEE itself. The process includes repair steps triggered by missing dependencies (e.g., libgoogle-perftools-dev). A successful klee --version smoke test confirms stage completion.

Stage 3: Project setup. KLEE requires LLVM bitcode. The agent pins fastfetch to release 2.54.0, builds it with wllvm for whole-program bitcode instrumentation, and runs extract-bc. Corrective actions include disabling link-time optimization (LTO), which the project enables by default but which is incompatible with whole-program bitcode extraction, and ensuring that llvm-link is discoverable. The presence of fastfetch.bc confirms stage completion.

Stage 4: Analysis run. The agent executes KLEE on the bitcode with conservative resource limits (Figure 1). The resulting evidence package comprises three components: (1) the KLEE invocation command targeting fastfetch.bc, (2) the output directory klee-out containing concrete test cases, and (3) runtime log messages confirming analysis progress. This package passes the LLM-as-judge validation. Premature termination (e.g., stopping after klee --version) would be rejected since no project-level artifacts exist.

4 Evaluation

We organize the evaluation around three research questions:

Table 3: LLMs used in evaluation (prices per 1M tokens).

Model	Input price	Output price	in/out limits
gpt-5-nano	\$0.05	\$0.40	400K / 128K
gpt-5-mini	\$0.25	\$2.00	400K / 128K
deepseek-v3.2	\$0.56	\$1.50	128K / 64K
gemini-3-flash	\$0.50	\$3.00	1M / 64K

- **RQ1 (Effectiveness):** How often do agents solve the Analysis-Bench tasks end-to-end under fixed budgets, and how do success rates vary across agent architectures, LLM backends, analysis tools, and project languages?
- **RQ2 (Qualitative analysis):** What distinguishes successful from unsuccessful runs? What are the dominant failure modes across all agents?
- **RQ3 (Efficiency and costs):** What are the resource requirements (time, cycles, LLM costs) of automated software analysis?

4.1 Experimental Setup

Configuration. We evaluate all agents on the full Analysis-Bench benchmark of 35 tasks (§2.2) using four LLM backends of varying capability and cost (Table 3). All agents share the same, per-task stopping limits: a maximum of 120 agentic cycles, a \$2 API cost cap, and a 5 h wall-clock timeout; reaching any limit interrupts the task and marks it as failed. For agents supporting retries (AnalysisAgent, ExecutionAgent), each retry counts as additional cycles within the same per-task budget. We repeat each configuration $n = 3$ times to account for non-determinism.

Success metrics. We assess task outcomes with two complementary metrics. First, a task is *self-validated* if the agent terminates and claims task completion. For AnalysisAgent, this includes passing its internal LLM-as-judge validation (§3.3); for baseline agents, it means the agent exited without error and reported success. Self-validated success may include false positives. To account for false positives, we inspect self-validated cases and call a task *manually verified* if it passes the manual validation process described in §2.3.

Statistical testing. We perform pairwise significance tests between AnalysisAgent and each baseline. For self-validated success, where both agents are evaluated on the same 35 tasks, we use McNemar’s one-sided exact test [39] to check whether AnalysisAgent can solve a task more often than the baselines in cases when one agent solves it and the other does not. For verified success, where only one run is available, we use Fisher’s one-sided exact test to check whether AnalysisAgent solves more tasks. All p -values are corrected for multiple comparisons ($3 \text{ baselines} \times 4 \text{ LLMs}$) using the Holm-Bonferroni method [25] at $\alpha=0.05$. We also test each agent’s absolute success rate against chance ($H_0: p \leq 0.5$) via exact binomial tests with 95% Wilson confidence intervals. Beyond p -values, we report Cohen’s h for effect size, odds ratios with 95% confidence intervals, and the Cochran-Mantel-Haenszel test to confirm that results hold across LLM backends.

4.2 RQ1: Effectiveness Across Agents and LLMs

Table 4 reports success rates by agent and LLM backend, discussed below. The discrepancy between self-validated and manually verified success shows the importance of manual verification, instead of only relying on LLMs to judge success.

Verified success. Manually verified success rates span a wide range across agent architectures: RAG-Agent averages only 10% across LLM backends (range 3–23%), showing that naive retrieval-based prompting is insufficient for multi-step analysis workflows. Mini-SWE-Agent, a general-purpose coding agent, reaches an average of 37% but exhibits high LLM sensitivity (9–63%), suggesting that without task-specific scaffolding, effectiveness depends heavily on the model’s capabilities. ExecutionAgent’s tool-aware adaptations raise the average to 57%, with less variation across LLMs (40–77%). AnalysisAgent achieves the highest manually verified success, averaging 79% across LLM backends (range 54–94%). It also achieves the best overall verified success rate of 94% with both Gemini-3-Flash and DeepSeek-V3.2 (33 of 35 tasks), demonstrating that purpose-built scaffolding can reliably handle the multi-step workflows required to install tools, build target projects, and produce meaningful analysis evidence. We highlight Gemini-3-Flash in the abstract because it also achieves the lowest mean time (36 minutes) among the two tied backends.

Statistical analysis. In short, AnalysisAgent’s advantage over all three baselines is large, consistent across all LLM backends, and statistically significant. Figure 2 visualizes the per-LLM success odds ratio of AnalysisAgent vs. other agents. We organize the statistical evidence around two questions:

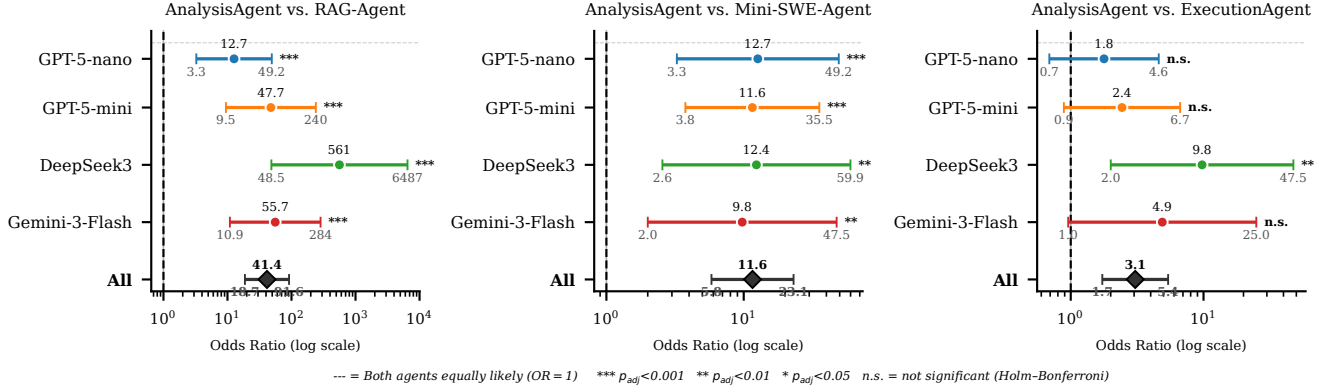
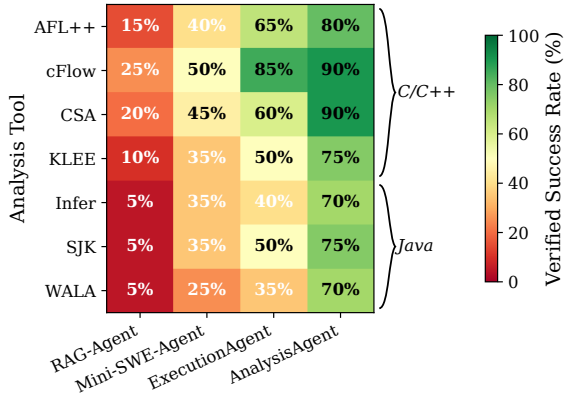
- (1) *Does AnalysisAgent outperform the baselines, and by how much?* Fisher’s exact test with Holm-Bonferroni correction yields $p_{\text{adj}} < 0.001$ for all three pairwise comparisons, pooled across LLM backends. The effect sizes are large: Cohen’s $h=1.55$ vs. RAG-Agent, $h=0.92$ vs. Mini-SWE-Agent, and $h=0.45$ vs. ExecutionAgent. The odds ratios are $\text{OR}=34.5$ [17.3, 68.5] vs. RAG-Agent, $\text{OR}=8.1$ [4.0, 16.2] vs. Mini-SWE-Agent, and $\text{OR}=2.7$ [1.6, 4.6] vs. ExecutionAgent; all 95% confidence intervals exclude 1.
- (2) *Does the advantage hold across all LLM backends?* The Cochran-Mantel-Haenszel test controls for LLM choice and confirms significance for all three baselines ($\chi^2_{\text{MH}}=139.8$, $p < 10^{-10}$; 43.3, $p=4.7 \times 10^{-11}$; 15.4, $p=8.8 \times 10^{-5}$), with common odds ratios 41.4, 11.2, 3.1.

Self-validated success. On self-validated success (McNemar’s exact test), AnalysisAgent significantly outperforms ExecutionAgent on all backends ($p_{\text{adj}} < 0.001$), but not RAG-Agent or Mini-SWE-Agent, since those agents almost always claim success (e.g., Mini-SWE-Agent on DeepSeek-V3.2 reports 100% success, but only 57% validated success). Furthermore, from Table 4, we observe that:

- *Self-validated success is inflated across all agents.* For AnalysisAgent with Gemini-3-Flash the gap is small (100% self-validated vs. 94% manually verified), but baselines exhibit much larger gaps, ordered $\text{RAG-Agent} > \text{Mini-SWE-Agent} > \text{ExecutionAgent}$. RAG-Agent and Mini-SWE-Agent lack explicit output validation and readily accept partial results as success; ExecutionAgent

Table 4: Self-validated and manually verified success rates by agent architecture and LLM backend. Self-validated rates are mean $\pm$ standard deviation over $n=3$ runs; verified rates are from manual review of a single run.

Agent	GPT-5-nano		GPT-5-mini		DeepSeek-V3.2		Gemini-3-Flash		Avg. Verified
	Self-val.	Verified	Self-val.	Verified	Self-val.	Verified	Self-val.	Verified	
RAG-Agent	73 $\pm$ 7%	9%	98 $\pm$ 1%	6%	49 $\pm$ 16%	3%	93 $\pm$ 5%	23%	10%
Mini-SWE-Agent	100 $\pm$ 0%	9%	100 $\pm$ 0%	20%	100 $\pm$ 0%	57%	87 $\pm$ 16%	63%	37%
ExecutionAgent	30 $\pm$ 37%	40%	63 $\pm$ 31%	54%	50 $\pm$ 3%	57%	86 $\pm$ 19%	77%	57%
AnalysisAgent	77 $\pm$ 21%	54%	97 $\pm$ 4%	75%	100 $\pm$ 0%	94%	100 $\pm$ 0%	94%	79%

**Figure 2: How many times more likely AnalysisAgent is to succeed compared to baselines, per LLM backend (circles) and pooled across all LLMs (diamonds). Values above 1 mean AnalysisAgent succeeds more often; 1 means equal odds.****Figure 3: Average verified success rate by analysis tool and agent, aggregated over all LLM backends.**

filters some false positives by requiring a working output script; AnalysisAgent reduces the gap further via an LLM-as-judge step.

- *Weaker models produce more false positives.* GPT-5-nano and GPT-5-mini are more prone to premature termination or misinterpreting partial progress as task completion, leading to inflated self-validated rates.
- *Baseline agents underutilize strong LLMs.* Even with Gemini-3-Flash, baselines show high self-validated but lower verified success, i.e., architectural limitations (stage mixing, poor error localization, premature termination) rather than model capability.

Tool and ecosystem patterns. We next stratify manually verified success rates by tool and language ecosystem. Figure 3 reports

verified success rates for each tool and agent. Across all agents, cflow achieves the highest average success rate (63%), followed by CSA (54%) and AFL++ (50%), while WALA has the lowest success rate (34%). WALA and Infer are the largest failure contributors (17.5% and 16.5% of total failures, respectively), followed by SJK (15.5%) and KLEE (15.2%), with cflow contributing the fewest (9.9%). At the ecosystem level, Java tasks account for 62% of all failures versus 38% for C/C++, and this gap persists across all agents, reflecting the complexity of Java toolchains (classpaths, bytecode generation, JVM attachment) and heavyweight whole-program analyses. We cannot fully disentangle tool complexity from project complexity as each ecosystem uses different target projects. The consistency of the gap across all agents suggests that tool-side factors dominate.

4.3 RQ2: Qualitative Analysis

We first describe success patterns observed in AnalysisAgent runs, then systematically categorize all failed runs across the four agents by manually inspecting cycle-level trajectories, identifying recurring failure patterns, and iteratively refining categories until all runs are classified. Finally, we merge similar categories to get at a concise taxonomy. For baseline qualitative analysis, also see §3.2.

4.3.1 Success Patterns. Successful AnalysisAgent runs share three characteristics: (i) early convergence on a working container (typically within 10 cycles), (ii) incremental stage-by-stage progress with targeted error recovery, and (iii) evidence production and validation at each stage.

Example: KLEE on masscan (DeepSeek-V3.2). The agent applies KLEE to masscan, a C port scanner, completing in 29 cycles (\$0.39,

Table 5: Analysis output statistics of successful Analysis-Agent runs (377/420). IQR=interquartile range (Q1–Q3) reduces outlier influence; median shown for metrics and time.

Tool	Metric	IQR	Median	Time (s)
AFL++	Code Coverage (%)	0.39–1.2	0.44	210
	Tested Inputs	19K–1.8M	230K	
	Exec Speed Per S	108–1.6K	569	
	Corpus Size Seeds	0–63	6	
KLEE	Test Cases Generated	1–32	2	17
	Completed Paths	0–6	1	
	Instructions Executed	4.3K–95K	13K	
CSA	Bugs Reported	0–24	0	144
cflow*	Output Lines	25–118	60	103
	Functions Listed	11–69	41	
Infer	Issues Reported	3–24	3	102
	Top: thread safety (283), resource leak (16), null deref (13)			
WALA	Call Graph Nodes	12K–157K	39K	204
	Cha Classes	7.2K–29K	9.5K	
	Call Graph Edges	148K–5.2M	4.3M	
SJK	Process Cpu (%)	0–1K	101	66
	Thread Count	10–59	15	

*cflow numbers reflect the truncated output the agent sees; full output files are larger.

12 min). Tool setup consumes 14 cycles resolving three dependency issues (TCMalloc, SQLite3, klee-uclibc) through pragmatic workarounds. The agent then identifies `massip_parse_ipv4` as a suitable entry point, prepares a KLEE driver with a symbolic buffer, compiles to LLVM bitcode, and runs KLEE with a 30-second limit. The run produces 28 completed test paths and 172 partial paths, confirmed by the LLM-as-judge validator via `.ktest` files.

Example: WALA on JMH (Gemini-3-Flash). The agent produces a WALA call graph for JMH in 30 cycles (\$0.34, 12 min). It selectively builds only the three required Java modules after the full build failed, then writes a CHA driver from scratch, resolving five API mismatches via JAR introspection. The final call graph contains 92K nodes and 2.9M edges.

4.3.2 Analysis Outputs. Beyond binary success, we examine *what* the analysis tools produce when agents succeed. Table 5 gives a summary for successful AnalysisAgent runs. We report per-metric interquartile ranges (IQR) to reduce the influence of outliers. Successful runs produce meaningful analysis evidence across all seven tools: Infer reports issues spanning thread safety, resource leaks, and null dereferences, while WALA constructs call graphs with a median of 39K nodes and 4.3M edges. However, for the dynamic tools, the outputs remain shallow under short analysis budgets given to the agent: AFL++ achieves a median coverage of only 0.44% and KLEE generates a median of two test cases. The agent is instructed to limit the runtime of dynamic analysis tools and typically sets a 30–180 s timeout per invocation, leaving little time for path exploration. Longer budgets would likely improve dynamic metrics. Static analyzers are less affected by budget constraints. CSA and Infer complete whole-program analysis in 102–144 s. We extracted analogous metrics for baseline agents, but their higher false-positive rates and less structured logs make these statistics less reliable; we include them in the replication package for reference.

4.3.3 Failure modes. We classify 182 failing trajectories (of 560 verified instances) into root-cause categories, excluding infrastructure crashes and missing logs. Categories are non-exclusive (28% have multiple), so percentages in Figure 4 may exceed 100%:

- *Docker/build failure* (67): The agent is stuck in a Docker-build loop, fails to install the analysis tool, or cannot compile the target project, never reaching the analysis stage.
- *Analysis tool misuse* (73): The analysis tool is installed, but the agent invokes it incorrectly (wrong API classes, stale imports, wrong flags or input formats) or cannot produce required prerequisites, such as LLVM bitcode or a fuzzing harness.
- *Malformed LLM output* (42): The LLM produces output that the agent framework cannot parse as a valid tool call, causing cycles to be consumed without any action.
- *Budget/time exhausted* (34): The iteration budget or wall-clock limit is exhausted, typically co-occurring with other categories.
- *Incorrect analysis result* (21): The agent completes execution and submits a result, but the output is rejected by the validator or found to be incorrect during manual validation.

Each agent exhibits a distinct failure profile (Figure 4). RAG-Agent and ExecutionAgent primarily fail during environment construction: RAG-Agent is dominated by Docker/build failures (60%), reflecting its tendency to exhaust all 120 Dockerfile-generation iterations without learning from repeated build errors; ExecutionAgent is dominated by malformed LLM output (50%), followed by Docker/build failures (20%). The malformed-output failures are concentrated on DeepSeek-V3.2, which frequently returns empty responses that the framework cannot parse as valid tool calls, silently consuming cycles without progressing past environment setup. Mini-SWE-Agent fails at output quality and tool usage: Incorrect analysis results (48%) and analysis tool misuse (44%) dominate, as the agent often completes the workflow but produces inadequate analysis results or misuses the tool. Finally, AnalysisAgent has largely solved environment setup (only 7% of its failures) and instead fails during tool invocation: Analysis tool misuse (50%) is the leading cause, driven by incorrect WALA API usage, failed Infer build integration, and KLEE bitcode extraction failures. Two recurring signatures stand out: 33% of failing runs involve compile-fix loops (3+ consecutive compilation errors without resolution) and 23% include repeated validation rejections (5+ rejections); these thresholds describe observed behavior, not tuned parameters.

Example: WALA on Closure (GPT-5-mini). As an example, consider AnalysisAgent tasked with producing a WALA call graph for Google’s Closure compiler. After resolving initial *Docker/build failures* (42 cycles switching base images and building WALA from source), the agent entered a prolonged *analysis tool misuse* loop: five cascading WALA API errors of a stale import, a static-vs-instance method confusion, a class-name format error, a missing application JAR in the analysis scope, and finally a JAR missing transitive dependencies. Each fix led to the next error, exemplifying the compile-fix loop pattern (observed in 33% of failing runs). The trajectory consumed three attempts, ~102 cycles, and \$1.37 of the \$2 per-task budget (including retries), showing how cascading errors drain resources.

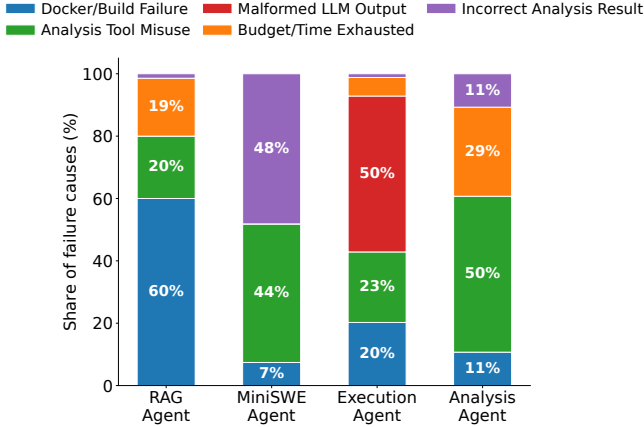


Figure 4: Root-cause failure categories by agent (%).

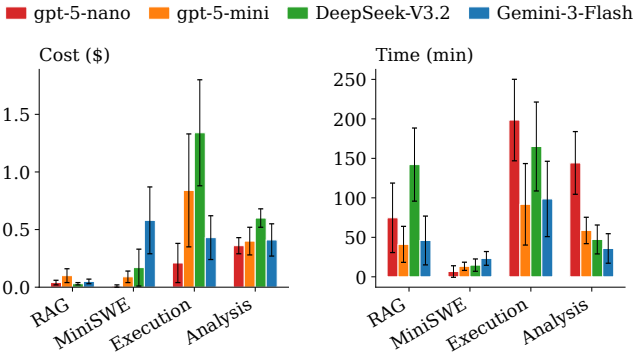


Figure 5: Resource consumption by agent and LLM backend.

4.4 RQ3: Efficiency and Costs

Figure 5 reports per-task resource consumption broken down by agent and LLM backend, covering monetary cost (computed from per-token pricing) and wall-clock time.

High variance across tasks. Figure 5 shows large standard deviations relative to the means across nearly all metrics. For example, ExecutionAgent with DeepSeek-V3.2 averages \$1.34 per task but with a standard deviation of \$0.46, and its token consumption varies substantially ($2351\text{ K} \pm 817\text{ K}$). This high variance implies that per-task cost is difficult to predict from the agent-model pair alone: Individual task characteristics (such as build-system complexity, dependency resolution difficulty, and whether the agent encounters early failures that trigger retries) appear to strongly influence the cost profile. From a practical standpoint, this means that budget planning for LLM-based analysis pipelines must account for particularly expensive tasks and not average only.

Capability-efficiency interaction. Model capability and efficiency are tightly coupled. The weakest backend (GPT-5-nano) consumes the most calls and wall-clock time (mean 78 calls and 144 minutes per task), despite having the lowest per-token pricing. Stronger backends substantially reduce interaction length and execution time: DeepSeek-V3.2 achieves a mean of 56 calls and 47 minutes per task, and Gemini-3-Flash reaches 36 minutes mean time.

Hence, token price alone is not a reliable proxy for per-task cost, as weaker models require more cycles and more tool executions.

Failure cost and model sensitivity. Failed runs are consistently more expensive than successful runs, with $2.77\times$ more cycles, $4.07\times$ longer duration, $1.27\times$ higher cost, and $1.76\times$ more retry attempts. The magnitude of this inflation varies by model: Gemini-3-Flash shows the highest sensitivity ($3.3\times$ more cycles on failure vs. success) despite the lowest absolute count on success (~ 30 cycles), while DeepSeek-V3.2 degrades more gracefully ($2.0\times$). In other words, a failing run does not just miss the goal: It consumes far more time and money than a successful one. Choosing a weaker or cheaper model does not save cost if it also fails more often. Similarly, reducing the timeout could curb the overhead of failed runs, but at the risk of terminating runs that would have eventually succeeded, potentially lowering overall success rates.

Language ecosystem differences. Java tasks are systematically more resource-intensive than C/C++ (e.g., mean 189 vs. 110 minutes and 269 vs. 176 calls for AnalysisAgent with GPT-5-nano; the same trend holds across all agent-model combinations), consistent with the higher failure rates reported in §4.2. We attribute this gap to the Java analysis toolchain rather than to the language itself. First, the Java tools in our benchmark carry heavier setup dependencies (e.g., Infer requires Buck, opam, OCaml, JDK, and Clang/LLVM), whereas several C/C++ tools have lighter dependency chains (e.g., cflow requires only Autotools). Second, the Java tools universally require a successful full build of the target project before analysis can begin: Infer and WALA operate on bytecode, and SJK must attach to a running JVM, while C/C++ tools, such as cflow and CSA, can operate more directly on source or integrate into the compilation step. Third, the Java tools demand more complex runtime orchestration. For instance, SJK requires launching the target as a background process and attaching the profiler, a coordination pattern absent from the C/C++ tools.

5 Discussion

5.1 Threats to Validity

Our benchmark covers seven tools and ten projects, which may not represent the full diversity of real-world analysis scenarios. In particular, (i) all target projects are command-line programs with well-defined entry points, whereas analyzing libraries may require additional harness construction; (ii) we focus on open-source tools, excluding commercial analyzers (e.g., Coverity) that may have different setup characteristics; and (iii) our project selection is limited to C/C++ and Java, leaving other ecosystems (e.g., Python, Rust) unexplored. Moreover, the 35 tasks are not statistically independent because multiple tasks share projects and tools. With $n=3$ repetitions, variance estimates are rough, particularly for weaker models (e.g., ExecutionAgent with GPT-5-nano: $30 \pm 37\%$), where non-determinism is highest; we report standard deviations for transparency but caution against over-interpreting them.

Manual validation was performed by a single primary validator who re-executed each agent setup and compared results against the reference artifacts; outlier cases were discussed among authors. While we did not measure formal inter-rater agreement, the validation relies on reproducible, artifact-based checks (file existence,

project-specific references, tool output comparison) rather than subjective judgment, and all verdicts, logs, and rubrics are included in the replication package for independent verification.

We measure success through tool-specific checks rather than numeric thresholds (e.g., we do not require a minimum number of KLEE paths or AFL++ corpus entries). This binary criterion evaluates whether the agent-produced setup is *correct and functional*, not whether the analysis itself is exhaustive. The low medians in Table 5 (e.g., AFL++ 0.4% coverage, KLEE 2 test cases) reflect short runtimes (30–60 s for dynamic tools), not setup quality; we re-run borderline setups with extended budgets to confirm functionality. For instance, we let two AFL++ setups run for several hours and observed crashes and hangs, confirming that the agent-produced harnesses and configurations are correct and that longer runtimes yield deeper analysis. A tiered metric (e.g., partial vs. full completion) is a natural extension, but evidence of a correct, functional setup is already a notable achievement given the task complexity.

5.2 Implications and Future Work

For practitioners. LLM agents can automate the deployment of analysis tools with high reliability when using strong models and structured agent architectures. For the best-performing configurations (AnalysisAgent with Gemini-3-Flash or DeepSeek-V3.2), the mean cost remains under \$1 per task (Figure 5), making this practical for CI/CD integration. Users should, however, incorporate evidence validation rather than relying on self-validated success.

For researchers. AnalysisBench can serve as a testbed for evaluating new agent architectures and LLM capabilities on environment-intensive tasks. Extending the benchmark to additional tools, languages, and project types (including libraries) is a natural next step. The most impactful next step is a controlled user study comparing agent-assisted setup time to fully manual setup, which would quantify the practical time savings for developers. Another promising direction is to explore adaptive agent strategies that detect early failure patterns (e.g., compile-fix loops) and dynamically switch strategies or reallocate budget to mitigate wasted resources. Finally, we envision AnalysisAgent enabling large-scale studies of analysis tools across many codebases.

6 Related Work

Environment construction and build reproducibility. Building software reliably and reproducibly is a well-known problem [37]. Several techniques to address this problem have been proposed, including a technique for fixing dependency errors to make Python builds reproducible [42] and a learning-based approach to localize and fix build errors in Java projects [56]. Reproducibility challenges affect not only project builds but also, e.g., bug datasets [69] and computational notebooks [59]. A related problem is to identify and install dependencies required to run code, e.g., addressed by DockerizeMe [26], which infers dependencies required to execute a Python code snippet without import errors. Our work targets multiple languages and build ecosystems, and it focuses on the broader task of applying analysis tools, not just building projects.

Agents for environment setup. ExecutionAgent introduced the idea of delegating the task of setting up an execution environment

of a given project to an LLM-based agent [6]. Other recent work further explores this idea [16, 29, 41, 63] and uses it as a first step toward creating SWE-bench-like [31] benchmarks of repository-level tasks [58, 63]. Similar to our work, these approaches use LLM-based agents to create a containerized environment and set up a given project inside it. In contrast, our work requires setting up not just the project but also a separate analysis tool with its own dependencies, and applying it to produce verifiable results. This dual requirement (tool installation, artifact preparation, analysis invocation) explains the higher failure rates vs. prior work [6]. Our evaluation directly compares to ExecutionAgent, showing AnalysisAgent outperforms it.

Other software engineering agents. LLM-based software engineering agents have attracted considerable attention in recent years. Issue solving and program repair are among the most prominent tasks, with notable examples including SWE-Agent [62], RepairAgent [5], AutoCodeRover [67], OpenHands [60], FixAgent [33], AgentCoder [30], Magis [55], MarsCode Agent [36], and Trae Agent [18]. Other tasks addressed by software engineering agents include generating issue-reproducing tests [1, 10, 43, 44], root cause analysis [46], and debugging computational notebooks [22]. Our work targets automated software analysis, introduces a new benchmark, and proposes an agent design tailored to this task.

Agent design and prompt engineering. ReAct [64] improves agent reasoning via interleaved reasoning and acting, and Reflexion [49] via verbal self-reflection. Our single-action cycle design aligns with the goal of immediate feedback. The staged structure of AnalysisAgent is complementary to these techniques and future work may integrate self-reflection into AnalysisAgent.

Benchmarks. AnalysisBench contributes a novel task and benchmark to the growing space of benchmarks for evaluating LLMs and LLM agents. Other benchmarks include SWE-bench [31] for issue solving, AgentBench [35] for interactive tasks, SUPER [4] for setting up and executing tasks from research repositories, NoCodebench [13] for documentation-based code editing, benchmarks for assessing code reasoning abilities [14] and code generation abilities [65], and several benchmarks for reproducing and replicating scientific results [4, 27, 50, 51]. In addition to benchmarking LLMs, prior work has proposed benchmarks for evaluating program analysis tools, e.g., fuzzers [24, 40]. While some existing work requires agents to set up environments and execute tasks, AnalysisBench focuses specifically on *automated software analysis*: the agent must install a separate analysis tool, prepare tool-specific prerequisites (e.g., LLVM bytecode, compilation databases), and validate that the tool produces meaningful analysis outputs on the target project. This dual setup requirement (tool + project) and the need for domain-specific output validation distinguish our benchmark from prior agent evaluation suites.

7 Conclusion

We introduce the task of automated software analysis and present AnalysisBench, a benchmark of 35 tool-project pairs, together with AnalysisAgent, a purpose-built agent that achieves 94% verified success. Our evaluation across four agent architectures and four LLM backends shows that task-specific architecture provides gains

that model scaling alone does not, that self-validated success is unreliable without evidence-based validation, and that tool-side usability (especially Java toolchains) remains a bottleneck that neither stronger models nor better scaffolding fully resolve.

8 Data Availability

We will release AnalysisBench, AnalysisAgent, and experimental data for future research (<https://doi.org/10.5281/zenodo.19348151>).

References

- [1] Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved? *CoRR abs/2412.02883* (2024). arXiv:2412.02883 doi:10.48550/ARXIV.2412.02883
- [2] Subarno Banerjee, Lazaro Clapp, and Manu Sridharan. 2019. NullAway: practical type-based null safety for Java. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo (Eds.). ACM, 740–750. doi:10.1145/3338906.3338919
- [3] Al Bessey, Ken Block, Benjamin Chelf, Andy Chou, Bryan Fulton, Seth Haller, Charles Henri-Gros, Asya Kamsky, Scott McPeak, and Dawson R. Engler. 2010. A few billion lines of code later: Using static analysis to find bugs in the real world. *Commun. ACM* 53, 2 (2010), 66–75.
- [4] Ben Bogin, Kejuan Yang, Shashank Gupta, Kyle Richardson, Erin Bransom, Peter Clark, Ashish Sabharwal, and Tushar Khot. 2024. SUPER: Evaluating Agents on Setting Up and Executing Tasks from Research Repositories. arXiv:2409.07440 [cs.AI] <https://arxiv.org/abs/2409.07440>
- [5] Islem Bouzenia, Premkumar T. Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*. IEEE, 2188–2200. doi:10.1109/ICSE55347.2025.00157
- [6] Islem Bouzenia and Michael Pradel. 2025. You Name It, I Run It: An LLM Agent to Execute Tests of Arbitrary Projects. *Proc. ACM Softw. Eng.* 2, ISSTA (2025), 1054–1076. doi:10.1145/3728922
- [7] Cristian Cadar, Daniel Dunbar, and Dawson R. Engler. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs. In *Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX, 209–224.
- [8] Cristiano Calcagno, Dino Distefano, Jeremy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O'Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. 2015. Moving Fast with Software Verification. In *Proceedings of the 7th International Conference on NASA Formal Methods (NFM'15)* (Pasadena, CA, USA).
- [9] Oliver Chang, Jonathan Metzman, Max Moroz, Martin Barbella, and Abhishek Arya. 2016. OSS-Fuzz: Continuous fuzzing for open source software. URL: <https://github.com/google/ossfuzz> (2016).
- [10] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambroner, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *arXiv preprint arXiv:2502.01821* (2025).
- [11] Clang Static Analyzer [n. d.]. Clang Static Analyzer. <https://clang-analyzer.lvm.org>.
- [12] Charlie Curtsinger and Emery D. Berger. 2015. Coz: finding code that counts with causal profiling. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*. 184–197.
- [13] Le Deng, Zhonghao Jiang, Jialun Cao, Michael Pradel, and Zhongxin Liu. 2025. Nocode-bench: A benchmark for evaluating natural language-driven feature addition. *arXiv preprint arXiv:2507.18130* (2025).
- [14] Elizabeth Dinella, Satish Chandra, and Petros Maniatis. 2024. CRQBench: A Benchmark of Code Reasoning Questions. arXiv:2408.08453 [cs.SE] <https://arxiv.org/abs/2408.08453>
- [15] LangChain Docs. 2026. Build a RAG agent with LangChain. <https://docs.langchain.com/oss/python/langchain/rag>.
- [16] Aleksandra Eliseeva, Alexander Kovrigin, Ilia Kholkin, Egor Bogomolov, and Yaroslav Zharov. 2025. EnvBench: A Benchmark for Automated Environment Setup. *CoRR abs/2503.14443* (2025). arXiv:2503.14443 doi:10.48550/ARXIV.2503.14443
- [17] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. {AFL++}: Combining incremental steps of fuzzing research. In *14th USENIX workshop on offensive technologies (WOOT 20)*.
- [18] Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchun Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, et al. 2025. Trae agent: An llm-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370* (2025).
- [19] GNU cflow [n. d.]. GNU cflow. <https://www.gnu.org/software/cflow/>.
- [20] Google. 2015. Error Prone: static analysis tool for Java. <http://errorprone.info/>.
- [21] Susan L. Graham, Peter B. Kessler, and Marshall K. McKusick. 1982. Gprof: A call graph execution profiler. In *SIGPLAN Symposium on Compiler Construction*. ACM, 120–126.
- [22] Konstantin Grotov, Artem Borzilov, Maksim Krivobok, Timofey Bryksin, and Yaroslav Zharov. 2024. Debug Smarter, Not Harder: AI Agents for Error Resolution in Computational Notebooks. *arXiv preprint arXiv:2410.14393* (2024).
- [23] Xue Han, Tingting Yu, and Michael Pradel. 2021. ConfProf: White-Box Performance Profiling of Configuration Options. In *ICPE '21: ACM/SPEC International Conference on Performance Engineering, Virtual Event, France, April 19-21, 2021*, Johann Bourcier, Zhen Ming (Jack) Jiang, Cor-Paul Bezemer, Vittorio Cortellessa, Daniele Di Pompeo, and Ana Lucia Varbanescu (Eds.). ACM, 1–8. doi:10.1145/3427921.3450255
- [24] Ahmad Hazimeh, Adrian Herrera, and Mathias Payer. 2020. Magma: A ground-truth fuzzing benchmark. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 4, 3 (2020), 1–29.
- [25] Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70.
- [26] Eric Horton and Chris Parnin. 2019. Dockerizeme: Automatic inference of environment dependencies for python code snippets. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 328–338.
- [27] Chuxuan Hu, Liyun Zhang, Yeji Lim, Aum Wadhvani, Austin Peters, and Daniel Kang. 2025. REPRO-BENCH: Can Agentic AI Systems Assess the Reproducibility of Social Science Research? *arXiv preprint arXiv:2507.18901* (2025).
- [28] Ruida Hu, Chao Peng, Jingyi Ren, Bo Jiang, Xiangxin Meng, Qinyun Wu, Pengfei Gao, Xinchun Wang, and Cuiyun Gao. 2024. A Real-World Benchmark for Evaluating Fine-Grained Issue Solving Capabilities of Large Language Models. *arXiv preprint arXiv:2411.18019* (2024).
- [29] Ruida Hu, Chao Peng, Xinchun Wang, and Cuiyun Gao. 2025. An LLM-based Agent for Reliable Docker Environment Configuration. *arXiv preprint arXiv:2502.13681* (2025).
- [30] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. 2024. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 [cs.CL]
- [31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.
- [32] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why don't software developers use static analysis tools to find bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 672–681.
- [33] Cheryl Lee, Chunqiu Steven Xia, Longji Yang, Jen tse Huang, Zhouxin Zhu, Lingming Zhang, and Michael R. Lyu. 2024. A Unified Debugging Approach via LLM-Based Multi-Agent Synergy. arXiv:2404.17153 [cs.SE] <https://arxiv.org/abs/2404.17153>
- [34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [35] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>
- [36] Yizhou Liu, Pengfei Gao, Xinchun Wang, Chao Peng, and Zhao Zhang. 2024. MarsCode Agent: AI-native Automated Bug Fixing. *arXiv preprint arXiv:2409.00899* (2024).
- [37] Yiling Lou, Zhenpeng Chen, Yanbin Cao, Dan Hao, and Lu Zhang. 2020. Understanding build issue resolution in practice: symptoms and fix patterns. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 617–628.
- [38] Yingwei Ma, Rongyu Cao, Yongchang Cao, Yue Zhang, Jue Chen, Yibo Liu, Yuchen Liu, Binhua Li, Fei Huang, and Yongbin Li. 2024. Lingma SWE-GPT: An Open Development-Process-Centric Language Model for Automated Software Improvement. *arXiv preprint arXiv:2411.00622* (2024).
- [39] Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996
- [40] Jonathan Metzman, László Szekeres, Laurent Simon, Read Sprabery, and Abhishek Arya. 2021. Fuzzbench: an open fuzzer benchmarking platform and service. In *Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 1393–1403.
- [41] Louis Milliken, Sungmin Kang, and Shin Yoo. 2025. Beyond pip install: Evaluating llm agents for the automated installation of python projects. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*.

- IEEE, 1–11.
- [42] Suchita Mukherjee, Abigail Almanza, and Cindy Rubio-González. 2021. Fixing dependency errors for Python build reproducibility. In *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*. 439–451.
 - [43] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. 2024. Code Agents are State of the Art Software Testers. arXiv:2406.12952 [cs.SE] <https://arxiv.org/abs/2406.12952>
 - [44] Noor Nashid, Islem Bouzenia, Michael Pradel, and Ali Mesbah. 2026. Issue2Test: Generating Reproducing Test Cases from Issue Reports. In *International Conference on Software Engineering (ICSE)*.
 - [45] Pat Rondon, Renyao Wei, José Cambronero, Jürgen Cito, Aaron Sun, Siddhant Sanyam, Michele Tufano, and Satish Chandra. 2025. Evaluating Agent-Based Program Repair at Google. In *47th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, SEIP@ICSE 2025, Ottawa, ON, Canada, April 27 - May 3, 2025*. IEEE, 365–376. doi:10.1109/ICSE-SEIP66354.2025.00038
 - [46] Devjeet Roy, Xuchao Zhang, Rashi Bhawe, Chetan Bansal, Pedro Henrique B. Las-Casas, Rodrigo Fonseca, and Saravan Rajmohan. 2024. Exploring LLM-Based Agents for Root Cause Analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024, Porto de Galinhas, Brazil, July 15-19, 2024*, Marcelo d'Amorim (Ed.). ACM, 208–219. doi:10.1145/3663529.3663841
 - [47] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspán. 2018. Lessons from building static analysis tools at google. *Commun. ACM* 61, 4 (2018), 58–66.
 - [48] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: a concolic unit testing engine for C. In *European Software Engineering Conference and International Symposium on Foundations of Software Engineering (ESEC/FSE)*. ACM, 263–272.
 - [49] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
 - [50] Zachary S Siegel, Sayash Kapoor, Nitya Nagdir, Benedikt Stroebl, and Arvind Narayanan. 2024. Core-bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363* (2024).
 - [51] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. 2025. PaperBench: Evaluating AI's Ability to Replicate AI Research. *arXiv preprint arXiv:2504.01848* (2025).
 - [52] Nick Stephens, John Grose, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2016. Driller: Augmenting Fuzzing Through Selective Symbolic Execution. In *23rd Annual Network and Distributed System Security Symposium, NDSS 2016, San Diego, California, USA, February 21-24, 2016*.
 - [53] Swiss Java Knife (SJK) [n. d.]. Swiss Java Knife (SJK). <https://github.com/aragozin/jvm-tools>.
 - [54] T. J. Watson Libraries for Analysis (WALA) [n. d.]. T. J. Watson Libraries for Analysis (WALA). <https://github.com/wala/WALA>.
 - [55] Wei Tao, Yucheng Zhou, Wenqiang Zhang, and Yu Cheng. 2024. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. *arXiv preprint arXiv:2403.17927* (2024).
 - [56] Daniel Tarlow, Subhdeep Moitra, Andrew Rice, Zimin Chen, Pierre-Antoine Manzagol, Charles Sutton, and Edward Aftandilian. 2020. Learning to Fix Build Errors with Graph2Diff Neural Networks. In *ICSE '20: 42nd International Conference on Software Engineering, Workshops, Seoul, Republic of Korea, 27 June - 19 July, 2020*. ACM, 19–20. doi:10.1145/3387940.3392181
 - [57] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie J. Hendren, Patrick Lam, and Vijay Sundareshan. 1999. Soot - a Java bytecode optimization framework. In *Conference of the Centre for Advanced Studies on Collaborative Research (CASCON)*. IBM, 125–135.
 - [58] Konstantinos Vergopoulos, Mark Niklas Müller, and Martin Vechev. 2025. Automated benchmark generation for repository-level coding tasks. *arXiv preprint arXiv:2503.07701* (2025).
 - [59] Jiawei Wang, Tzu-yang Kuo, Li Li, and Andreas Zeller. 2020. Assessing and restoring reproducibility of Jupyter notebooks. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 138–149.
 - [60] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741 [cs.SE] <https://arxiv.org/abs/2407.16741>
 - [61] Guoqing (Harry) Xu, Matthew Arnold, Nick Mitchell, Atanas Rountev, and Gary Sevitsky. 2009. Go with the flow: profiling copies to find runtime bloat. In *Conference on Programming Language Design and Implementation (PLDI)*. ACM, 419–430.
 - [62] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html
 - [63] John Yang, Kilian Leret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025. SWE-smith: Scaling Data for Software Engineering Agents. *arXiv preprint arXiv:2504.21798* (2025).
 - [64] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.
 - [65] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Tao Xie, and Qianxiang Wang. 2024. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models. In *ICSE*.
 - [66] Michal Zalewski. 2013. American Fuzzy Lop (AFL). <https://lcamtuf.coredump.cx/afl/>. <https://lcamtuf.coredump.cx/afl/>
 - [67] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 1592–1604. doi:10.1145/3650212.3680384
 - [68] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. 2024. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems* 36 (2024).
 - [69] Hao-Nan Zhu and Cindy Rubio-González. 2023. On the Reproducibility of Software Defect Datasets. In *ICSE*.